ИТМО

Веб как платформа, жизненный цикл HTTP-запроса

Преподаватель курса

Минтус Евгений Андреевич

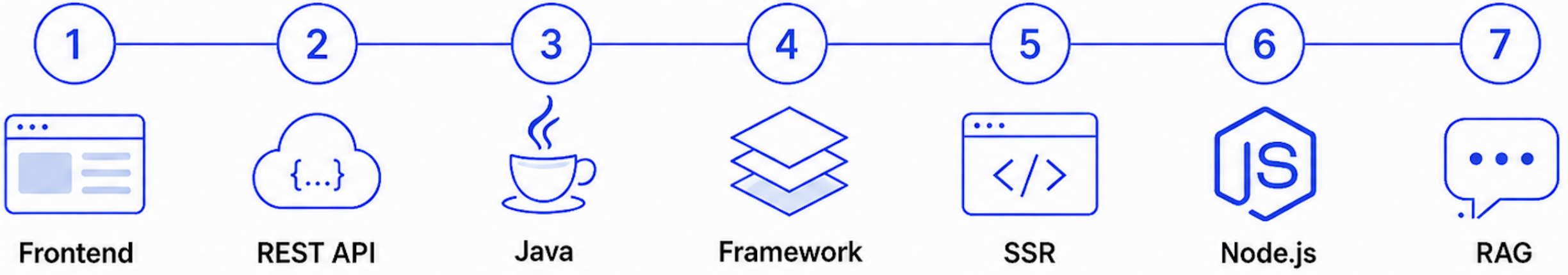
Web • Backend • Data & ML Engineering

На курсе будем разбирать веб не как набор технологий, а как целую систему - от браузера до базы данных.

Структура практической части курса

3 практика и лектора

7 лабораторных



Telegram

Что сегодня будет в лекции

URL → DNS → TCP/TLS → HTTP → Server → Response

01 **Веб как платформа**
интернет, веб, клиент и сервер

02 **Путь запроса**
URL, DNS, TCP/TLS и HTTPS

03 **HTTP изнутри**
request, headers, body и status codes

04 **REST и CRUD**
как API работает с ресурсами

05 **DevTools**
Network, timing, CORS и диагностика

Я просто ввел адрес…

Что реально запускает строка https://example.com?

В браузере кажется:

https://example.com

Нажал Enter → страница открылась

Но между этими двумя действиями проходит целая цепочка сетевых и HTTP-шагов.

Сегодня разложим этот путь по слоям

Я просто ввел адрес…

Что реально запускает строка https://example.com?

В браузере кажется:

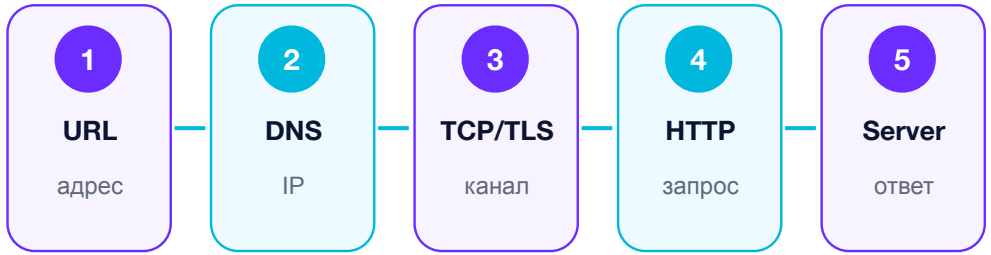
https://example.com

Нажал Enter → страница открылась

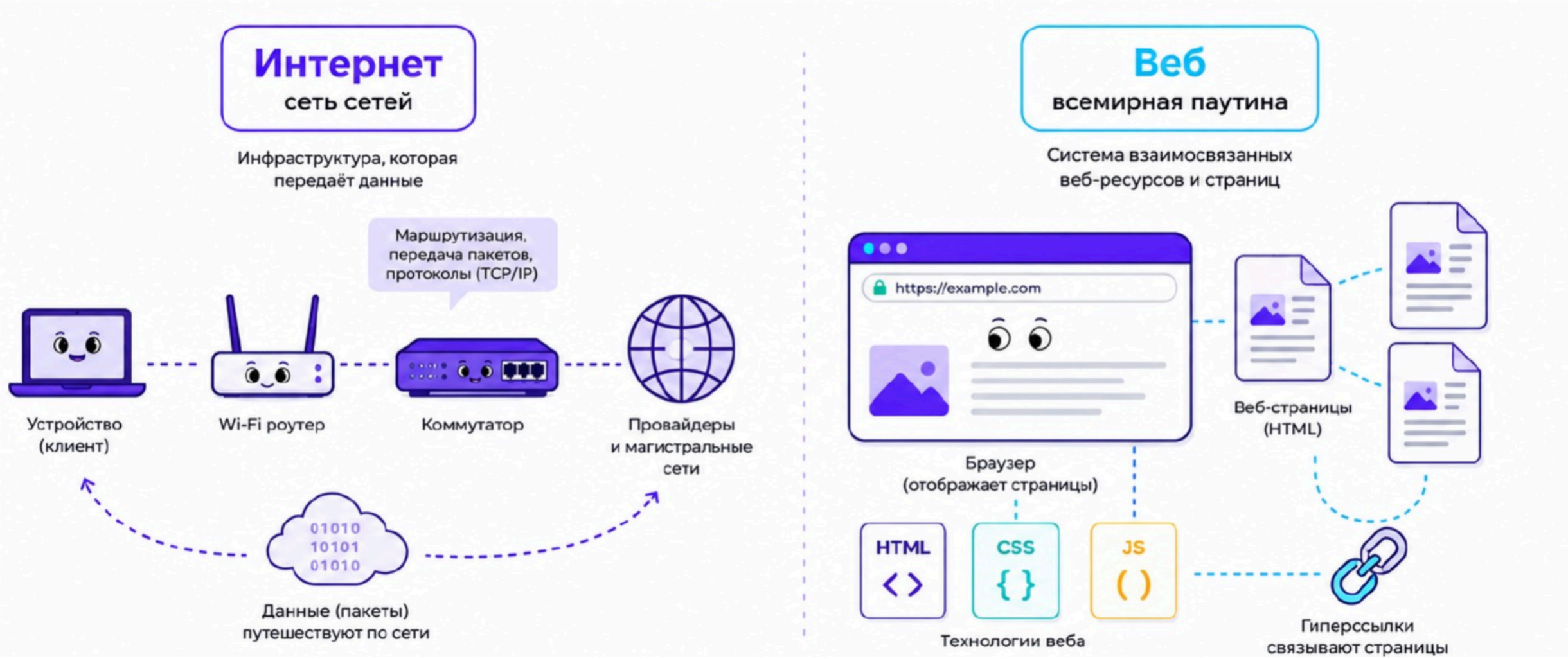
Но между этими двумя действиями проходит целая цепочка сетевых и HTTP-шагов.

Сегодня разложим этот путь по слоям

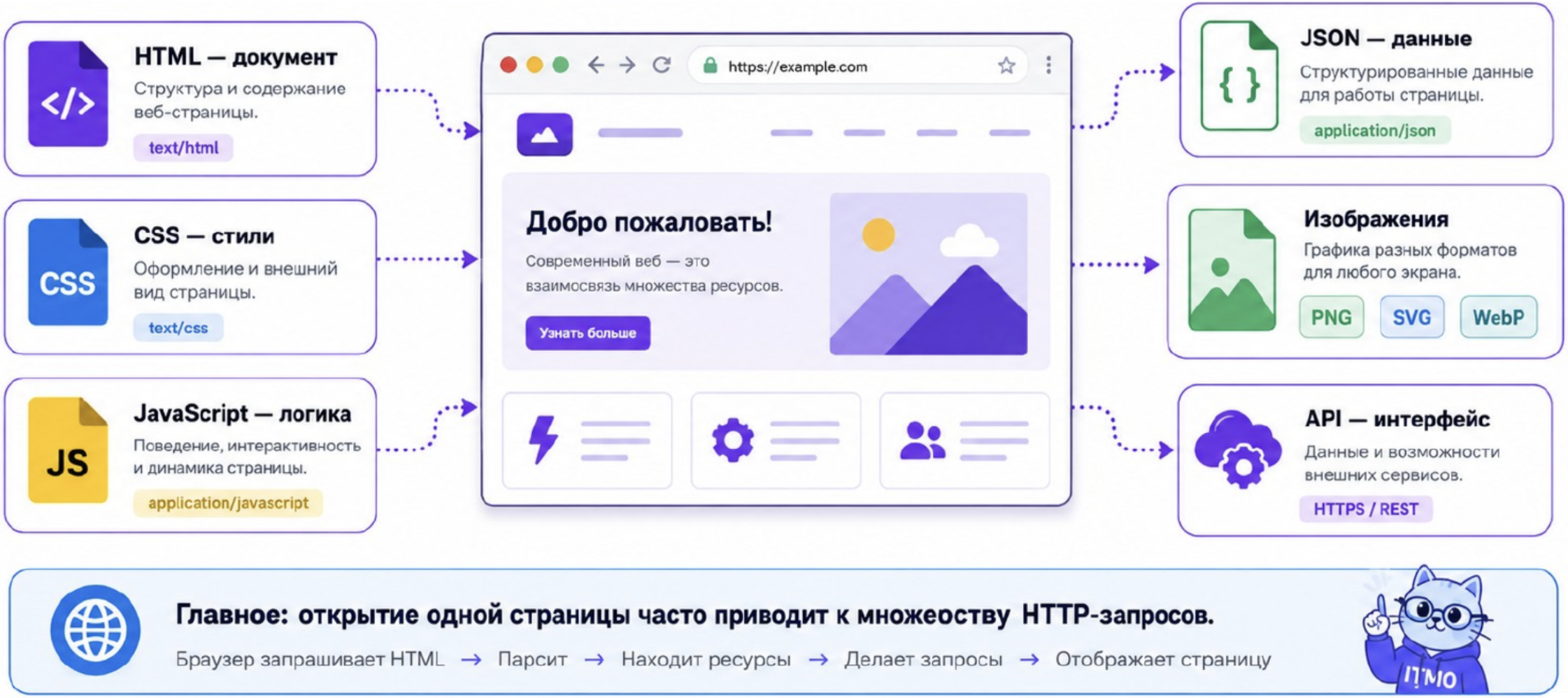
На самом деле запускается целая цепочка действий



Интернет ≠ Веб



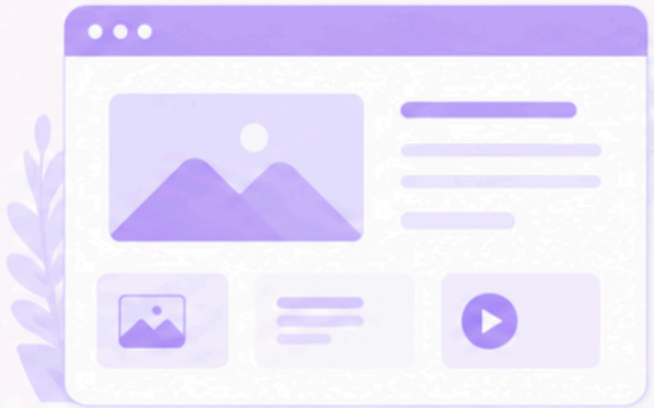
Веб - это связанные ресурсы



Веб-сайт, Веб-приложение, API

Веб-сайт

Контент и страницы для человека

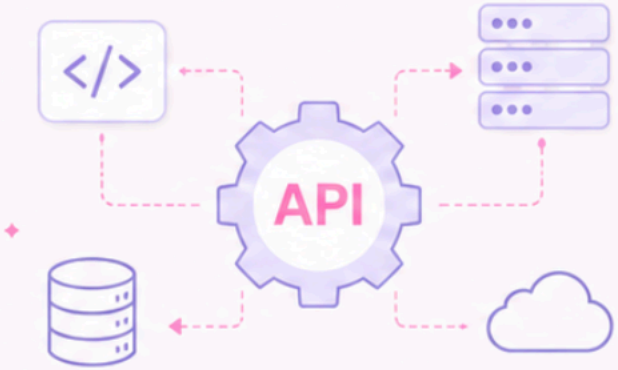


Веб-приложение

Интерактивный интерфейс и прикладная логика

API

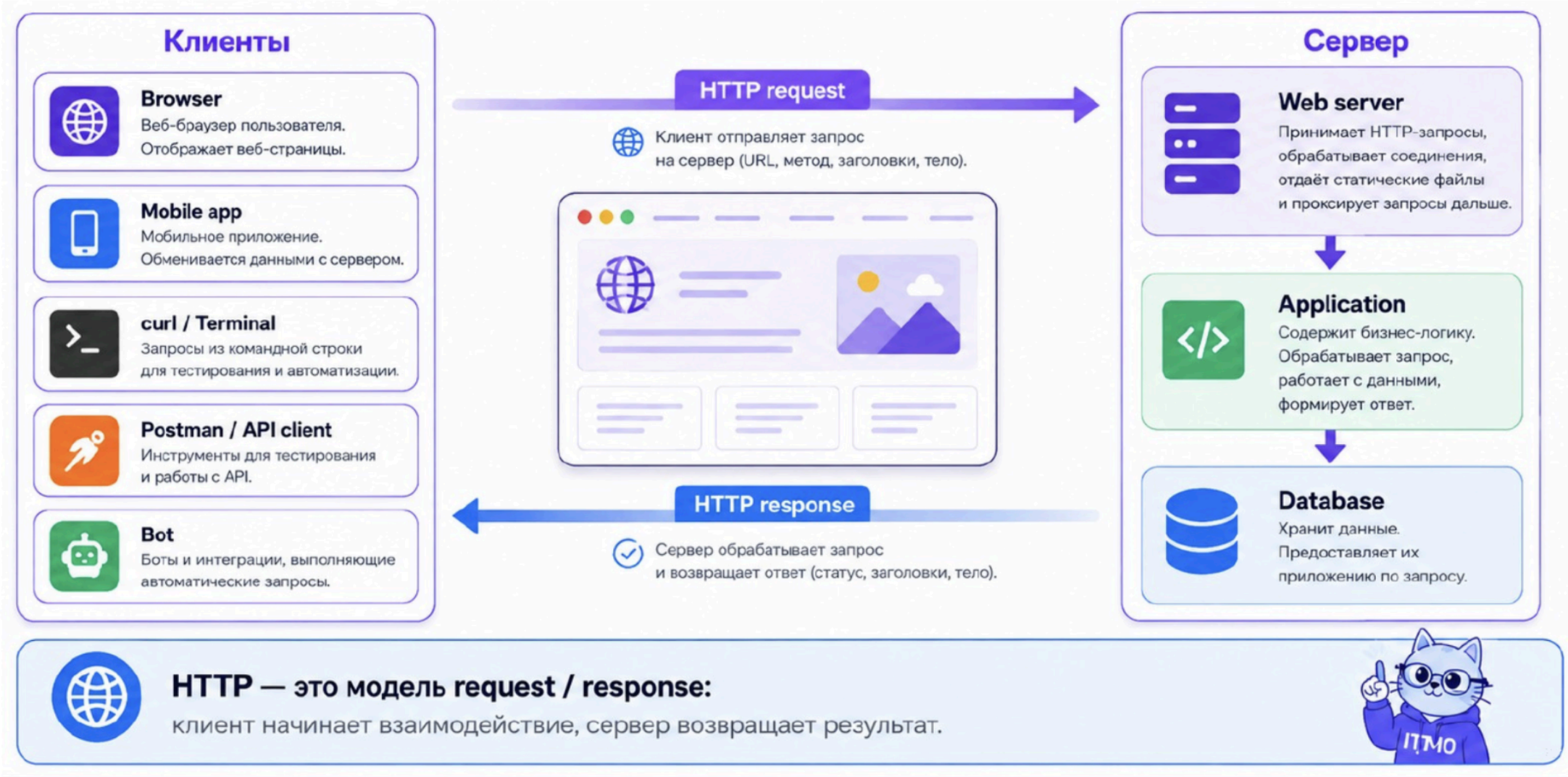
Интерфейс для взаимодействия программ



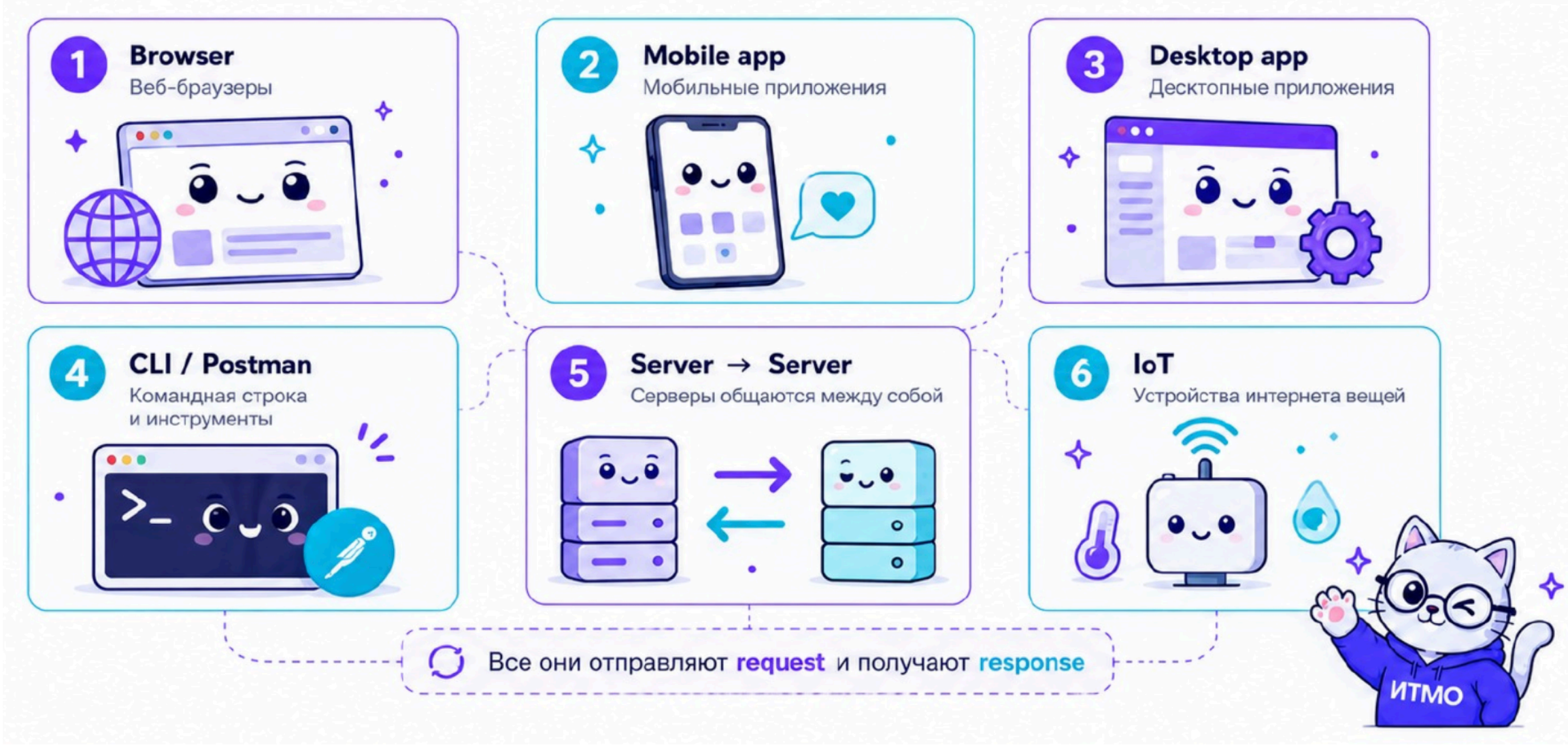
Все три могут пользоваться HTTP. Главное различие — кто основной потребитель: человек или программа.

Кто участвует в веб-взаимодействии

Клиент и сервер



Кто может быть клиентом?



Сервер это роль, а не волшебная коробка

Сервер может быть...

Одним процессом
Например, простой HTTP-сервис.

Набором сервисов
Каждый отвечает за свою задачу.

Контейнером в кластере
Масштабируется горизонтально.

Узлом в системе
Работает вместе с другими узлами.

В каком смысле говорят «сервер»?

Роль
Сторона, которая принимает запросы и возвращает ответ клиенту.

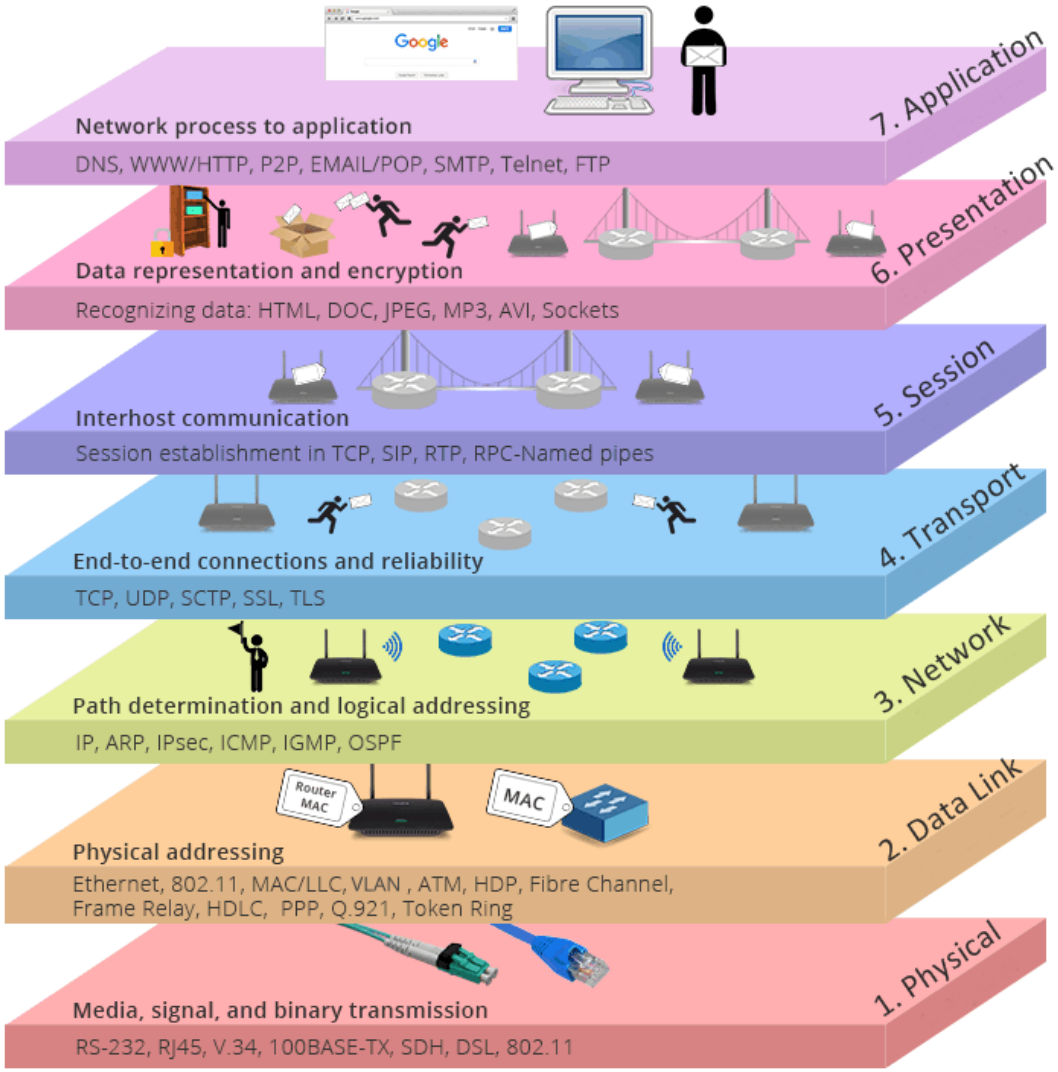
Программа / процесс
Приложение, которое слушает порт и обрабатывает запросы.

Машина / хост
Физический сервер, виртуальная машина или облачный инстанс, где запущены сервисы.

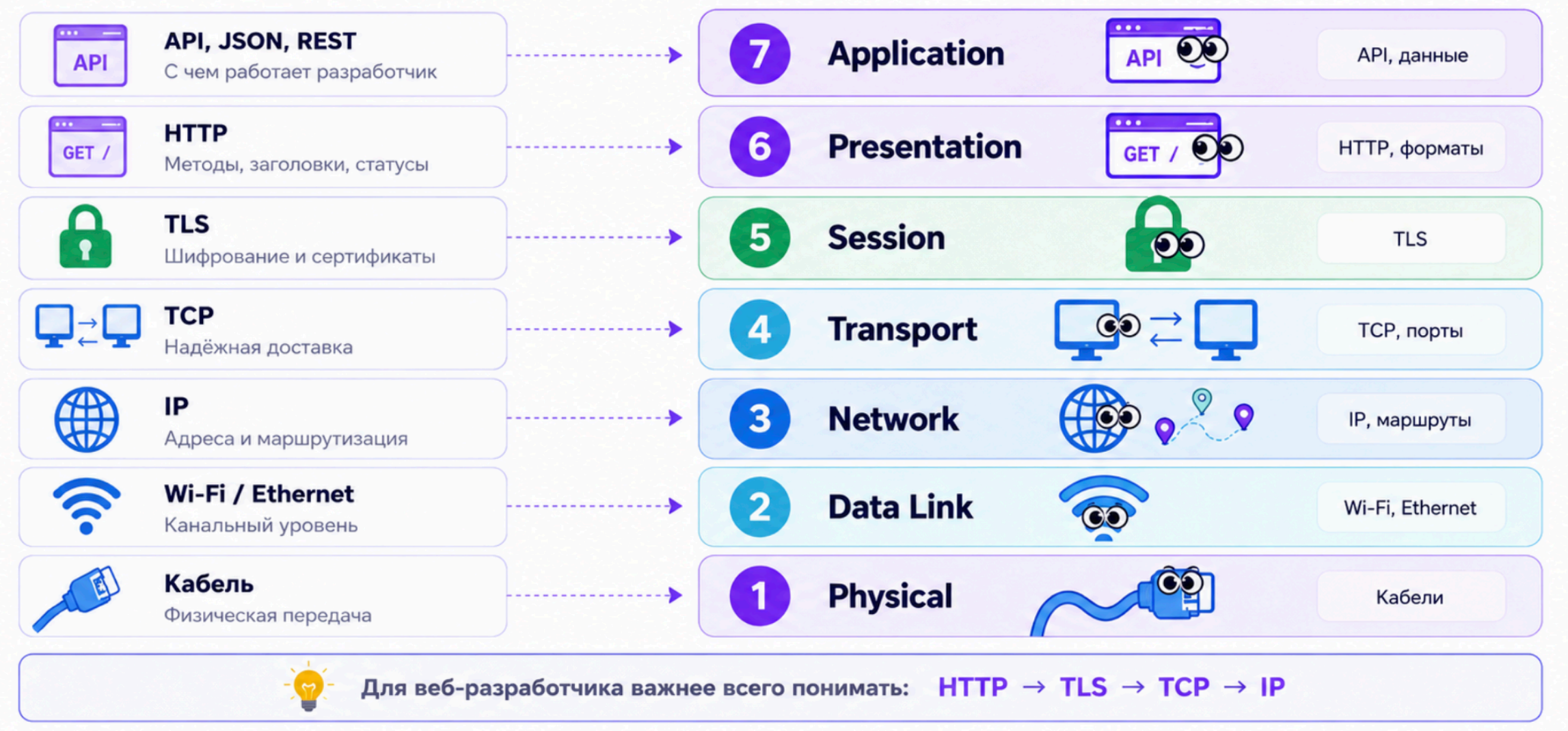
Сервис / узел системы
Один из компонентов backend-системы: API, auth, очередь, БД и др.

Контейнер — это способ упаковки и запуска приложения.

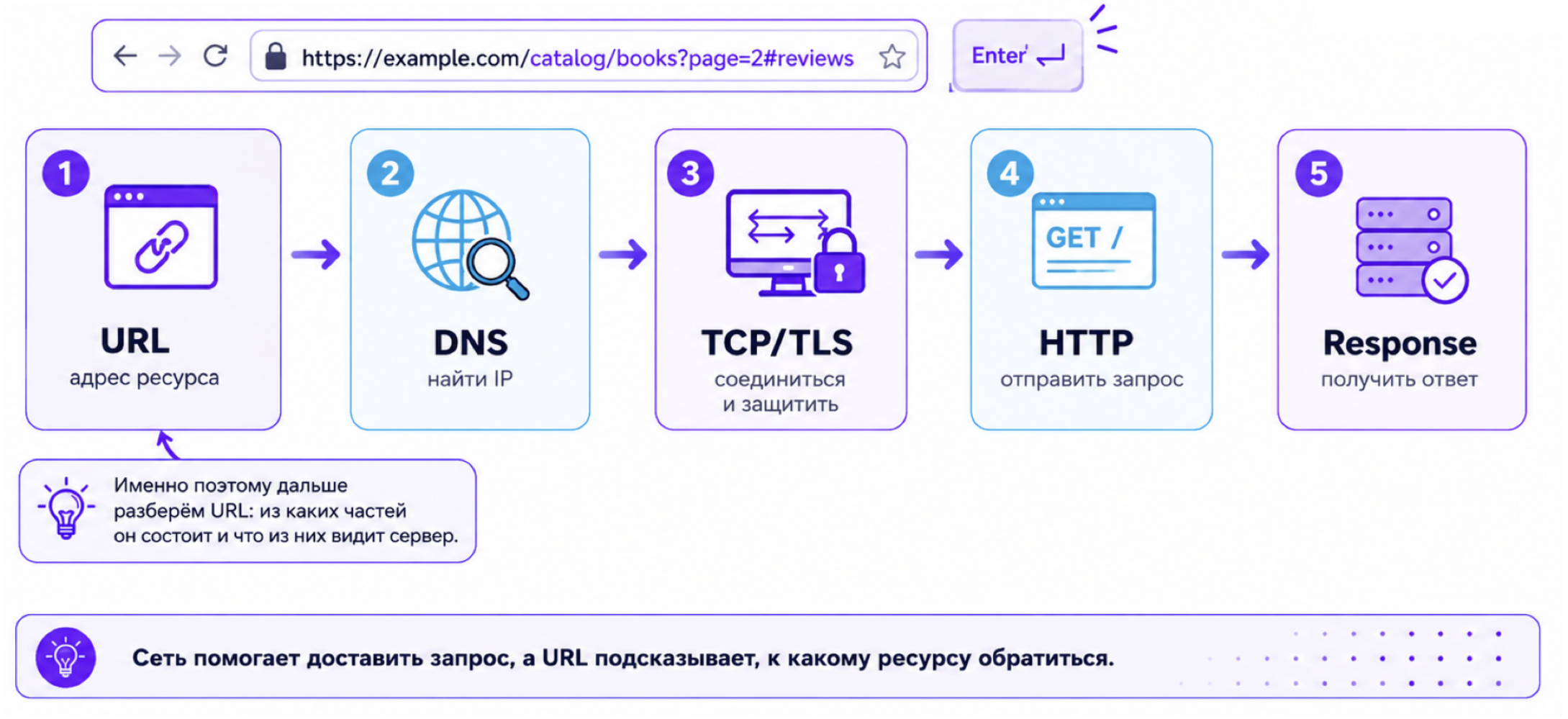
Модель OSI


Google
Network process to application
DNS, WWW/HTTP, P2P, EMAIL/POP, SMTP, Telnet, FTP
7. Application
Data representation and encryption
Recognizing data: HTML, DOC, JPEG, MP3, AVI, Sockets
6. Presentation
Interhost communication
Session establishment in TCP, SIP, RTP, RPC-Named pipes
5. Session
End-to-end connections and reliability
TCP, UDP, SCTP, SSL, TLS
4. Transport
Path determination and logical addressing
IP, ARP, IPsec, ICMP, IGMP, OSPF
3. Network
Router MAC
MAC
Physical addressing
Ethernet, 802.11, MAC/LLC, VLAN , ATM, HDP, Fibre Channel,
Frame Relay, HDLC, PPP, Q.921, Token Ring
2. Data Link
Media, signal, and binary transmission
RS-232, RJ45, V.34, 100BASE-TX, SDH, DSL, 802.11
1. Physical

Модель OSI для веб-разработчика



С чего начинается путь запроса?



Абсолютный и относительный URL

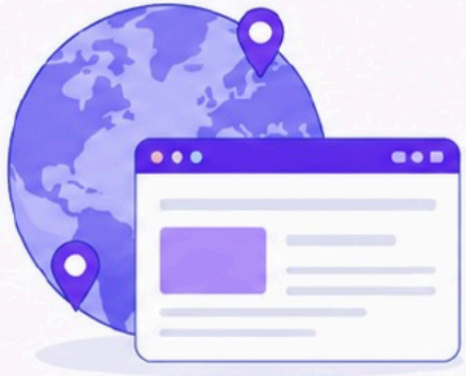
Абсолютный URL

Полный адрес ресурса

https://example.com/catalog/books

Протокол (HTTPS) | Домен (пример: example.com) | Путь к ресурсу (/catalog/books)

- ✓ Содержит протокол и домен
- ✓ Работает из любой точки интернета
- ✓ Не зависит от текущего местоположения



Относительный URL

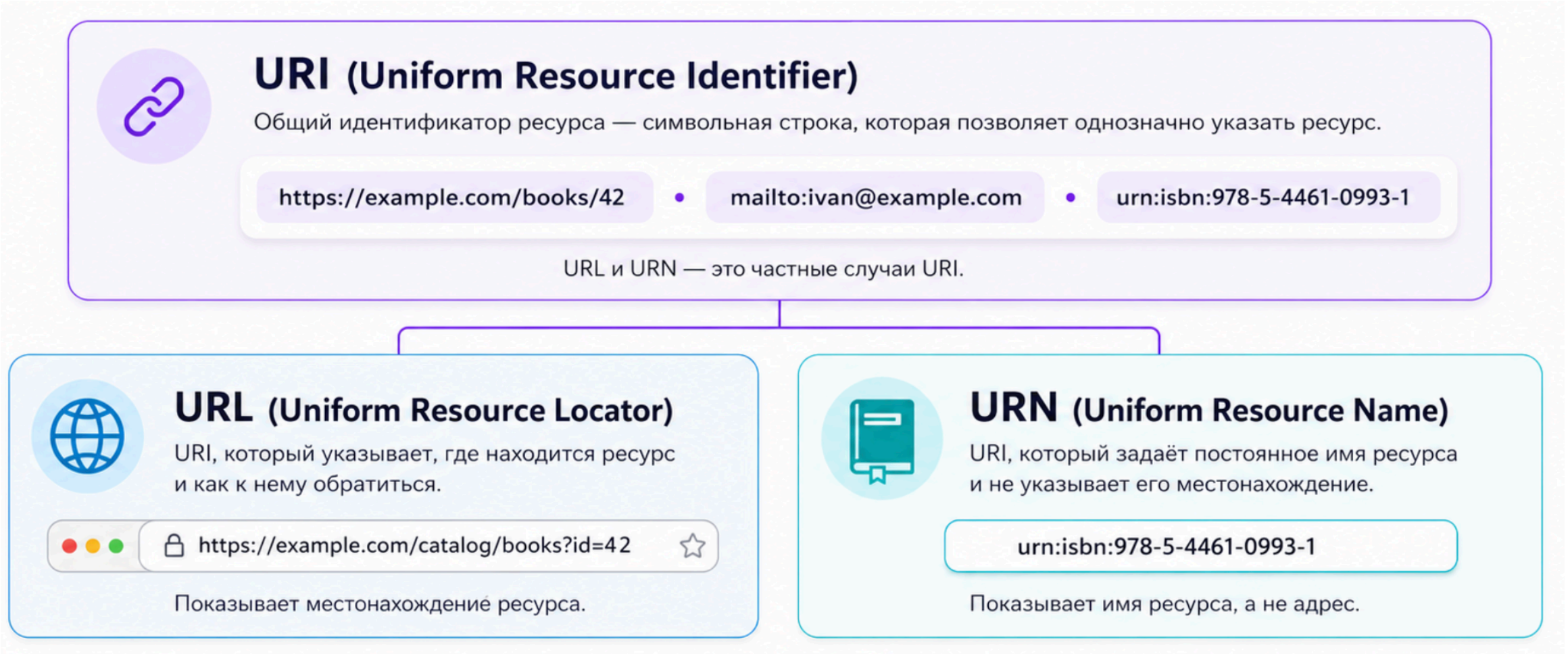
Путь относительно текущего ресурса

Пример 1	/catalog/books	От корня сайта (тот же домен)
Пример 2	../images/logo.svg	На уровень вверх и в папку images

- ✓ Короче и удобнее внутри сайта
- ✓ Зависит от текущего адреса страницы

Правило: браузер сам преобразует относительный URL в абсолютный внутри текущего контекста.

URI, URL, URN


URI (Uniform Resource Identifier)
Общий идентификатор ресурса — символьная строка, которая позволяет однозначно указать ресурс.
https://example.com/books/42
mailto:ivan@example.com
urn:isbn:978-5-4461-0993-1
URL и URN — это частные случаи URI.
URL (Uniform Resource Locator)
URI, который указывает, где находится ресурс и как к нему обратиться.
https://example.com/catalog/books?id=42
Показывает местонахождение ресурса.
URN (Uniform Resource Name)
URI, который задаёт постоянное имя ресурса и не указывает его местонахождение.
urn:isbn:978-5-4461-0993-1
Показывает имя ресурса, а не адрес.

Анатомия URL

URL — это адрес ресурса.
Браузеры знают, как его читать!

https:// example.com /catalog/books ?page=2 #reviews

scheme / схема
Протокол доступа. Чаще всего **https** (безопасный).

host / хост
Доменное имя сервера, на котором расположен ресурс.

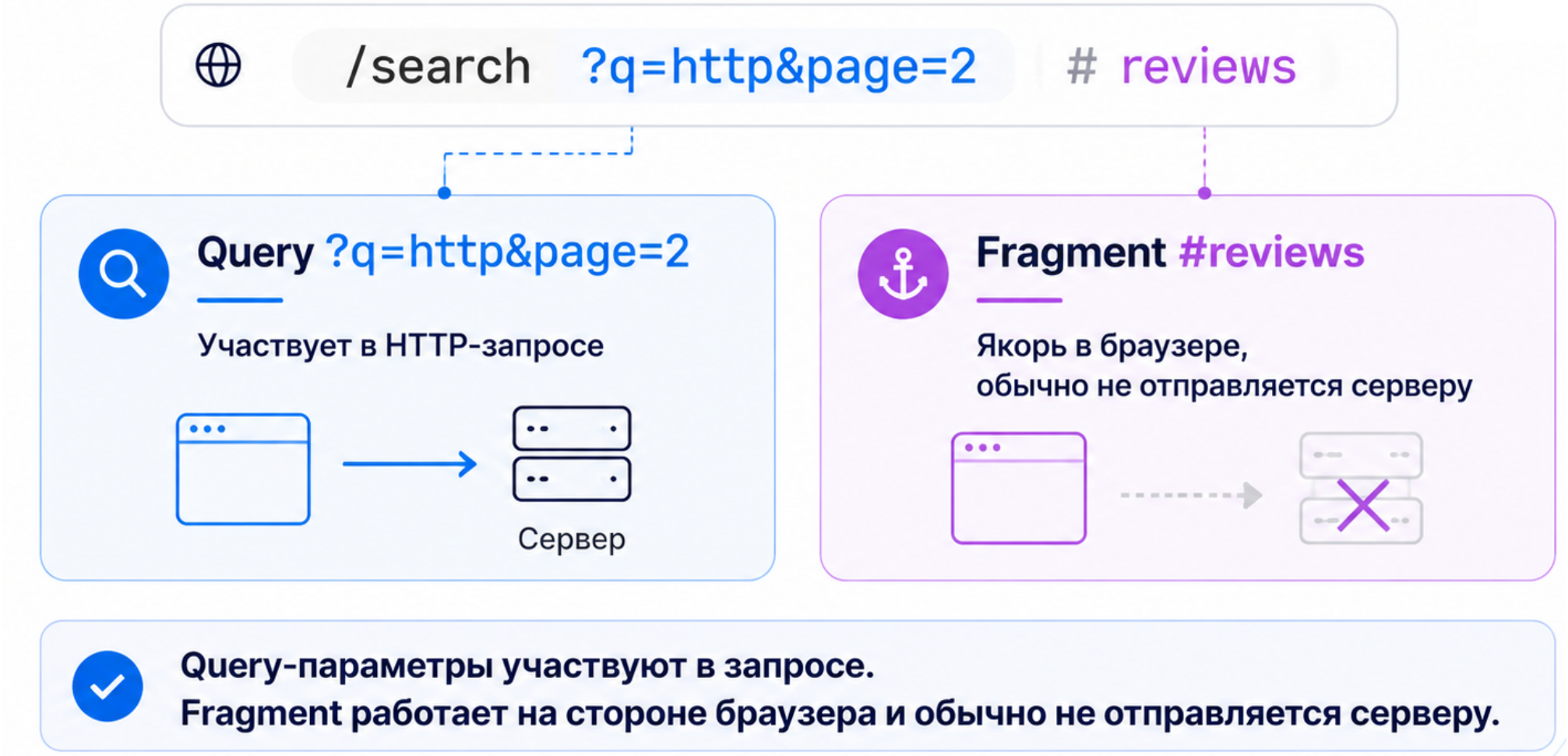
path / путь
Путь к ресурсу на сервере.

query / запрос
Параметры запроса. Используются для передачи данных.

fragment / фрагмент
Якорь на странице. Не отправляется на сервер.

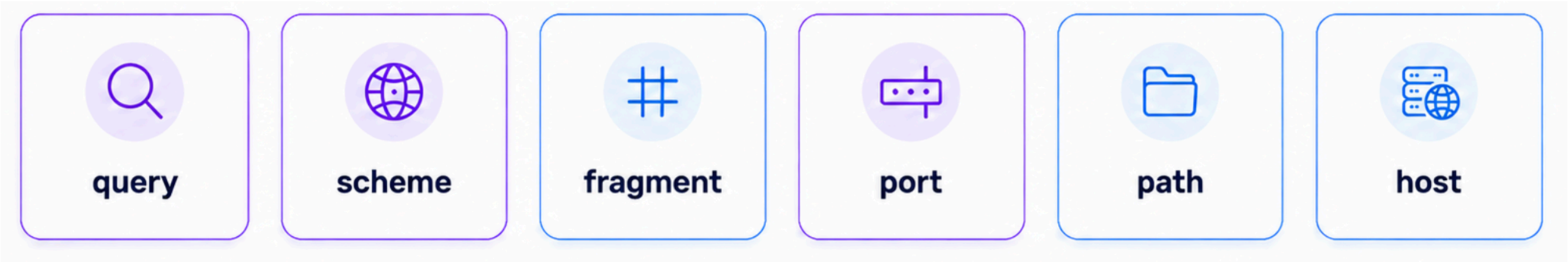
Важно!
Сервер получает только схему, хост, путь и параметры запроса.
Фрагмент обрабатывается браузером на стороне клиента.

Query и fragment: что увидит сервер?



Мини-проверка: разберите URL

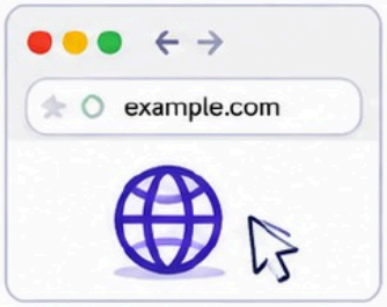
https://shop.example.com:443/catalog/phones?brand=acme&sort=price#specs



Зачем нужен DNS?

ИТМО

1 Вы вводите доменное имя

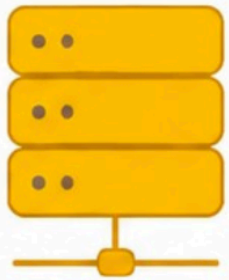


В браузере или приложении вы вводите адрес сайта example.com

2 Проверка локального кеша

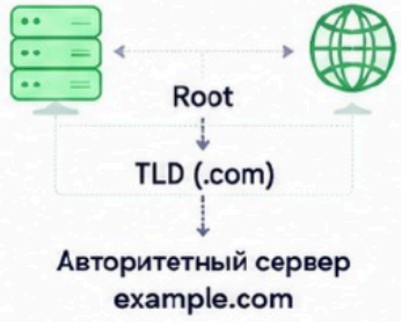
Операционная система проверяет локальный кеш. Если запись есть — использует её.

3 Запрос к рекурсивному DNS-резолверу



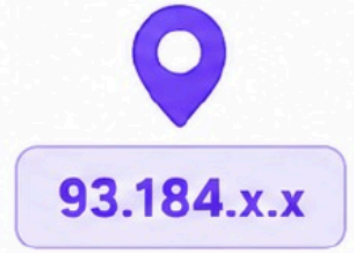
Если в кеше нет ответа, запрос отправляется к рекурсивному DNS-резолверу (обычно у вашего провайдера или публичному DNS).

4 Поиск в иерархии DNS



Резолвер последовательно обращается к серверам DNS: корневым (.), доменным верхнего уровня (.com), затем к авторитетному серверу домена example.com, чтобы получить данные.

5 Получение IP-адреса



Авторитетный сервер возвращает IP-адрес. Резолвер передаёт его приложению, и соединение устанавливается.

DNS связывает доменные имена и сетевые адреса.

Имя, которое легко запомнить человеку → Запрос через DNS → Поиск по иерархии → IP-адрес для сети → Доступ к сайту

Зачем нужен DNS?

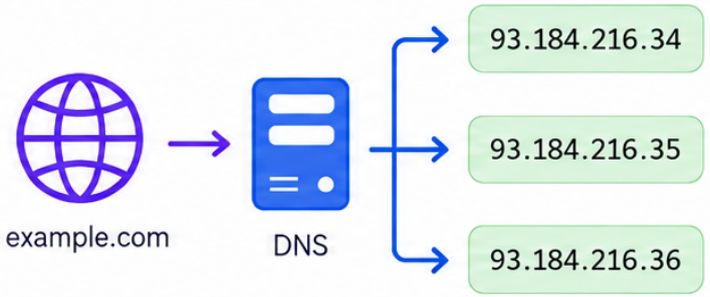
1 DNS-кэш

Ответы сохраняются, чтобы
не спрашивать снова и снова

Кэш = быстрее и меньше нагрузка

2 Один домен — несколько IP-адресов

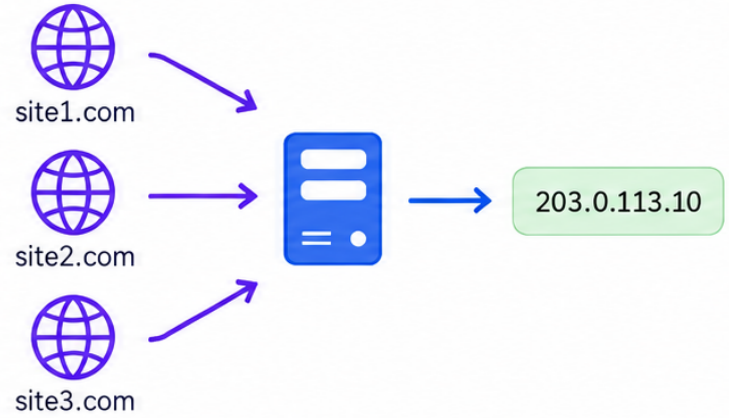
Для отказоустойчивости и баланса нагрузки



Клиент получает один из IP

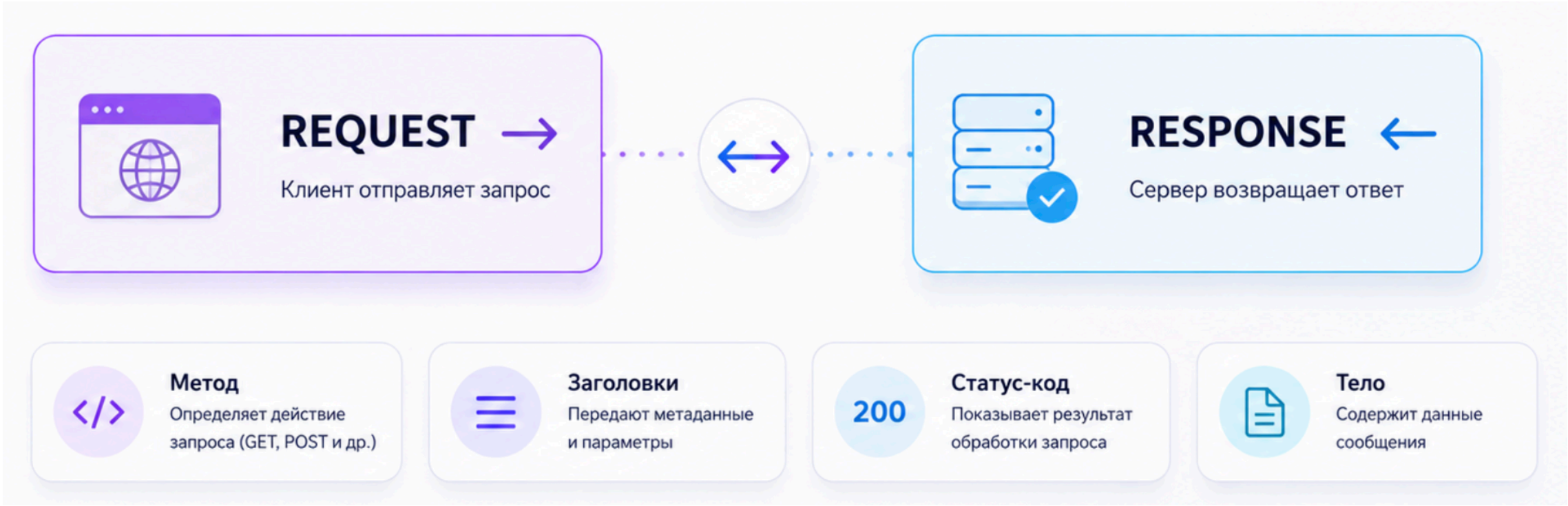
3 Несколько доменов — один IP-адрес

Виртуальный хостинг (SNI + Host)

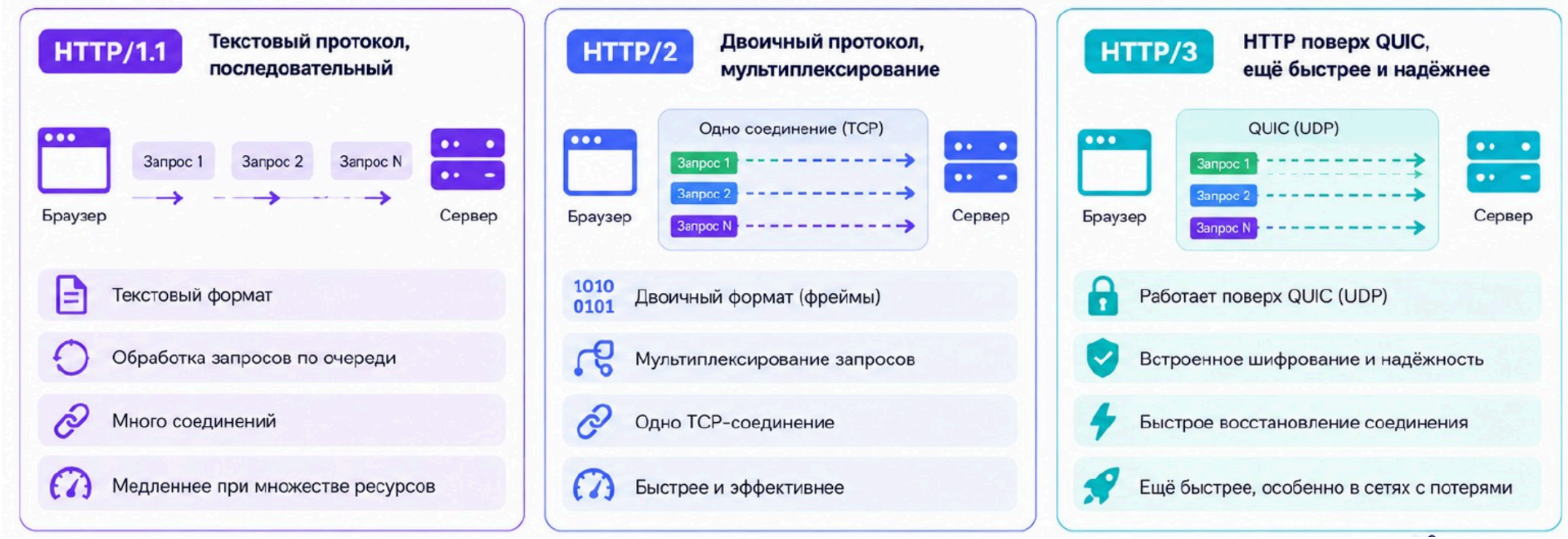


Экономия IP, проще масштабировать

Что такое HTTP?



Версии HTTP


HTTP/1.1
Текстовый протокол, последовательный
Браузер
Запрос 1
Запрос 2
Запрос N
Сервер
Текстовый формат
Обработка запросов по очереди
Много соединений
Медленнее при множестве ресурсов
HTTP/2
Двоичный протокол, мультиплексирование
Браузер
Одно соединение (TCP)
Запрос 1
Запрос 2
Запрос N
Сервер
1010 0101
Двоичный формат (фреймы)
Мультиплексирование запросов
Одно TCP-соединение
Быстрее и эффективнее
HTTP/3
HTTP поверх QUIC, ещё быстрее и надёжнее
Браузер
QUIC (UDP)
Запрос 1
Запрос 2
Запрос N
Сервер
Работает поверх QUIC (UDP)
Встроенное шифрование и надёжность
Быстрое восстановление соединения
Ещё быстрее, особенно в сетях с потерями

Что можно передавать по HTTP

ИТМО

1 HTML

Разметка веб-страницы, которая отображается в браузере.

2 JSON

Структурированные данные, часто используются в API.

3 TEXT

Обычный текст: письма, логи, конфиги, ответы сервера.

4 IMAGE

Картинки в форматах PNG, JPG, SVG и других.

5 VIDEO

Видеофайлы и потоки для воспроиз-ведения онлайн.

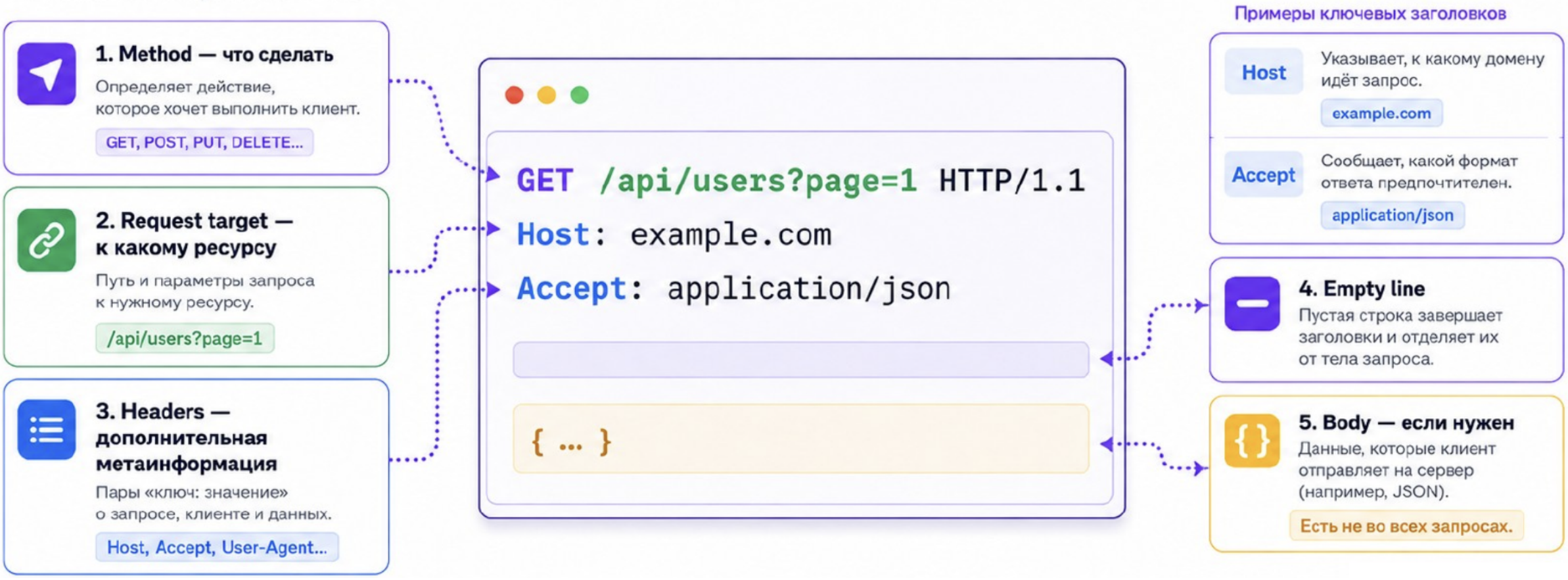
6 FILE

Любые файлы: архивы, PDF, документы, таблицы и т.д.

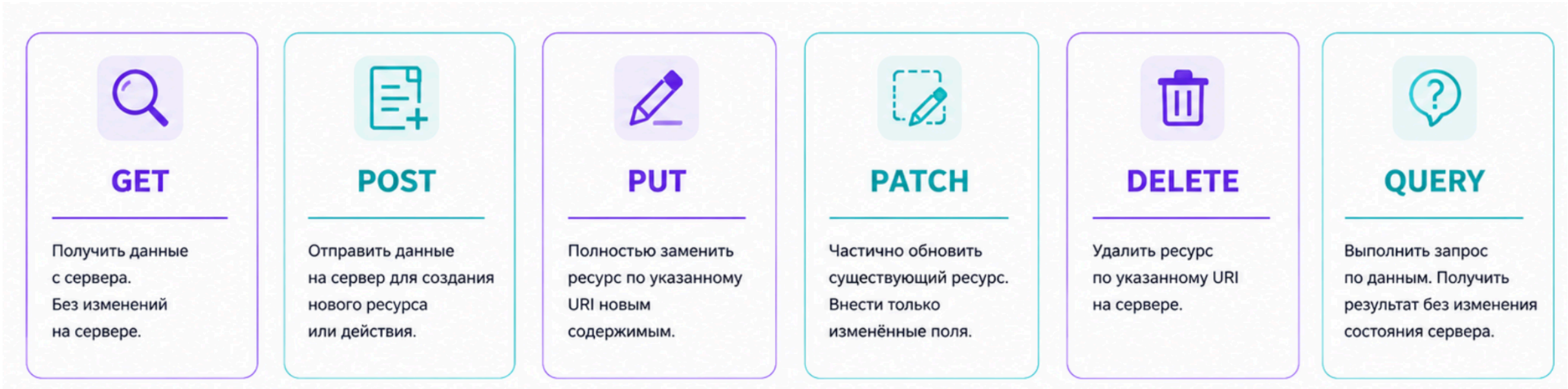
7 API data

Данные для приложений и сервисов (REST, RPC и др.).

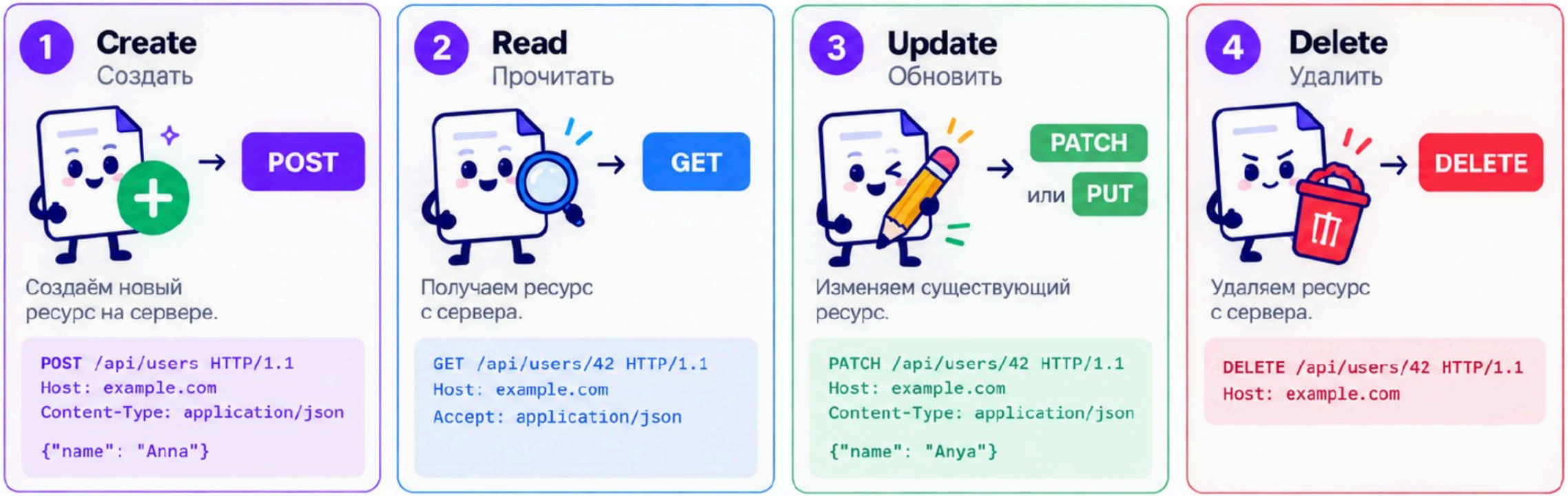
Из чего состоит HTTP запрос



HTTP methods



Связь между CRUD и HTTP



CRUD - это Create, Read, Update, Delete

HTTP QUERY и query parameters URL разные вещи!

query parameters в URL

```
GET /products?brand=acme&maxPrice=100
```

brand и maxPrice — это query-параметры, часть URL после знака ?.

vs

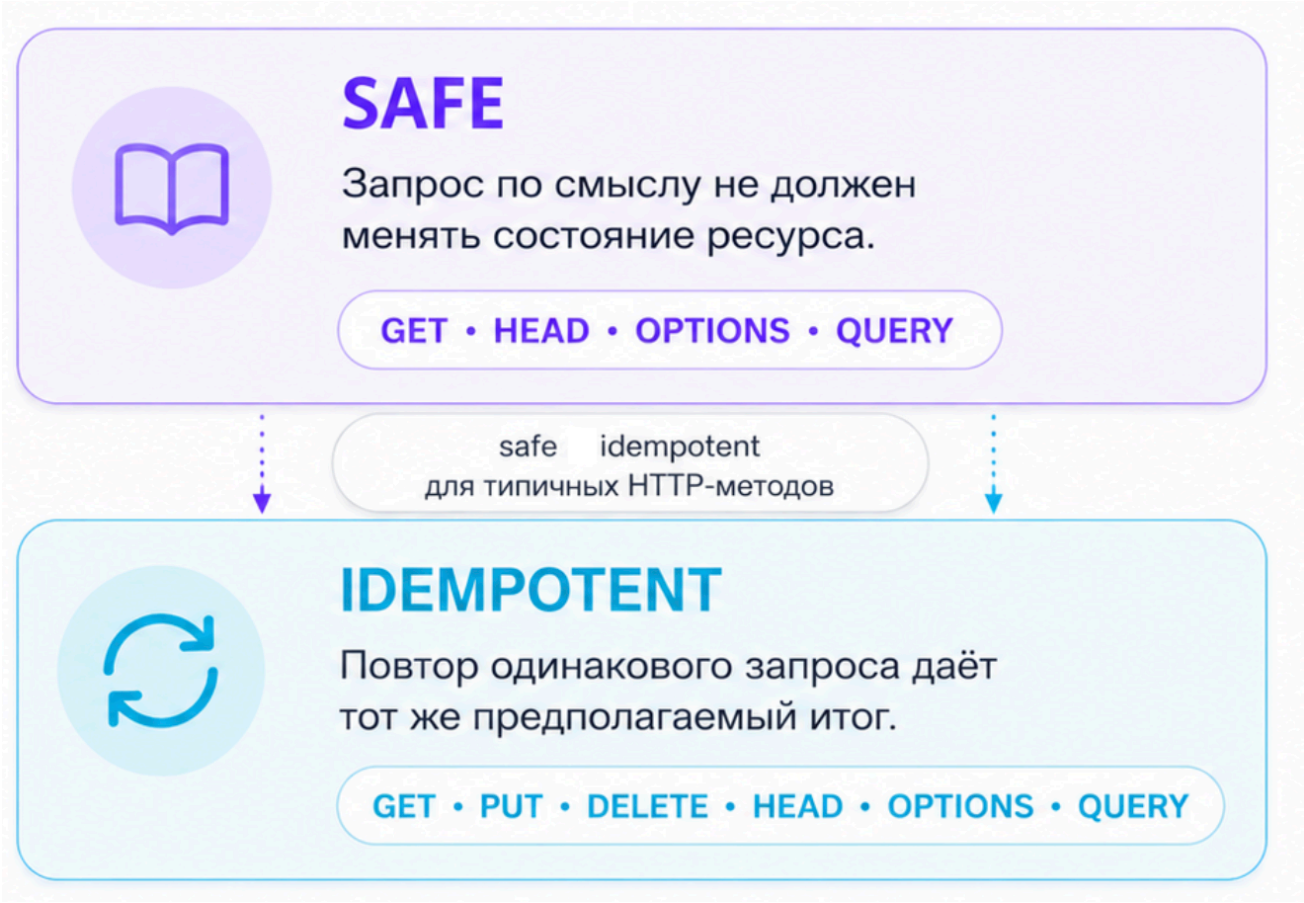
QUERY — HTTP method

```
QUERY /products HTTP/1.1
Content-Type: application/json

{ "brand": "acme" }
```

QUERY — отдельный HTTP-метод: параметры передаются в теле запроса, а не в URL.

Два свойства методов



POST чаще всего не safe и не idempotent: повтор может создать второй ресурс.

Headers: метаданные HTTP-сообщения

Request headers

Host
Домен и порт сервера, к которому направлен запрос.

Accept
Какие типы контента клиент готов получить в ответ.

Content-Type
В каком формате передано тело запроса.

Authorization
Данные для аутентификации (например, токен).

Cookie
Сохранённые данные о сессии пользователя.

User-Agent
Информация о клиенте (браузер, ОС, устройство).

Response headers

Content-Type
Формат тела ответа (JSON, HTML, изображение и др.).

Content-Length
Размер тела ответа в байтах.

Set-Cookie
Устанавливает cookie на стороне клиента.

Cache-Control
Управляет кешированием ответа на клиенте и промежуточных узлах.

Location
Адрес для перенаправления клиента (редирект).

HTTP-ответ: статус, заголовки, тело

Структура HTTP-ответов

HTTP-версия
Версия протокола, которую использует сервер.

Статус-код
Краткий результат запроса: успех, ошибка, перенаправление и т.д.

Заголовки (Headers)
Служебная информация о ответе: тип данных, кэш, язык, куки и др.

Тело (Body)
Основные данные ответа: JSON, HTML, текст, изображение и т.д.

```
HTTP/1.1 200 OK

Content-Type: application/json
Cache-Control: no-store
Date: Thu, 22 May 2025 12:34:56 GMT
Content-Length: 42

{
  "id": 123,
  "name": "Алиса"
}
```

Группы статус-кодов

1xx — Информационные
Запрос получен, продолжайте.
Примеры: 100, 101, 102

2xx — Успешные
Запрос успешно принят, понят и обработан.
Примеры: 200, 201, 204, 206

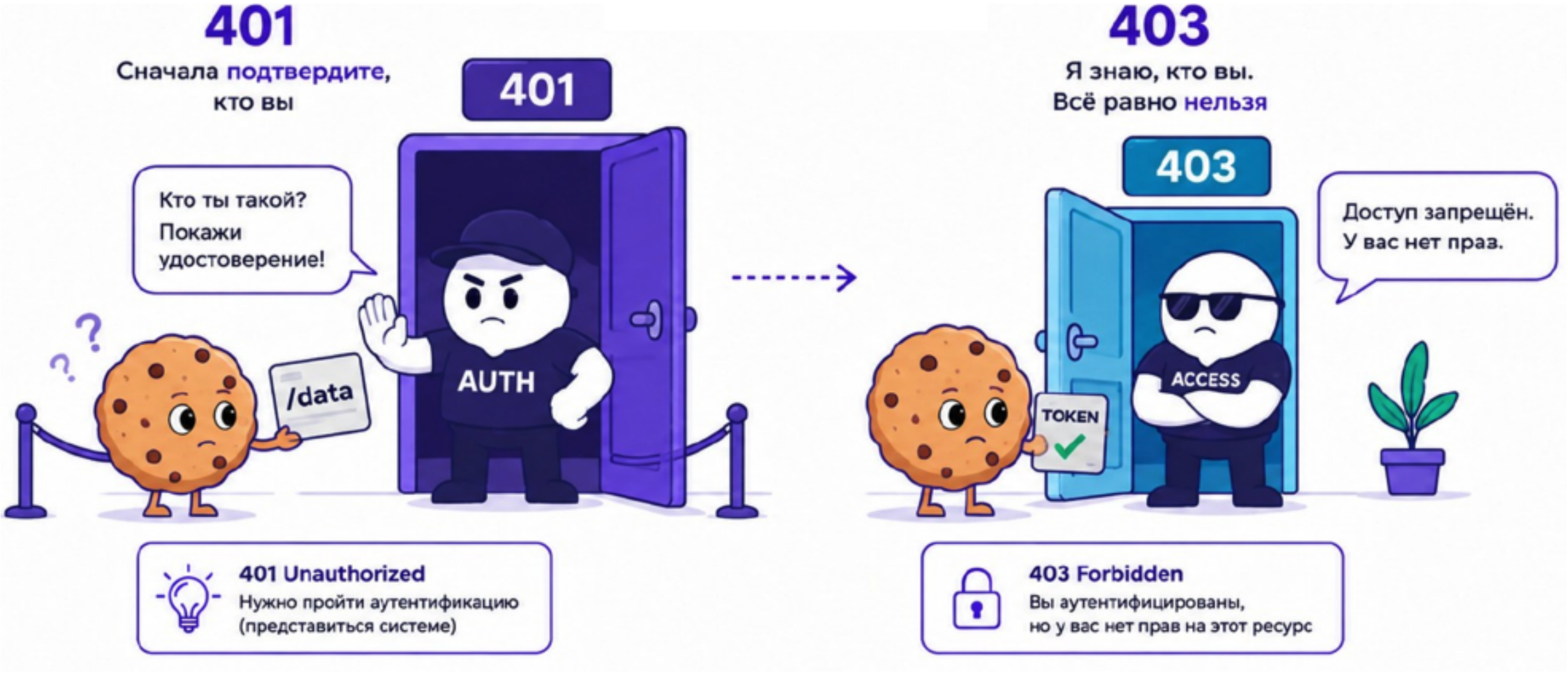
3xx — Перенаправления
Нужно выполнить дополнительное действие.
Примеры: 301, 302, 304, 307

4xx — Ошибки клиента
Проблема в запросе или правах доступа.
Примеры: 400, 401, 403, 404

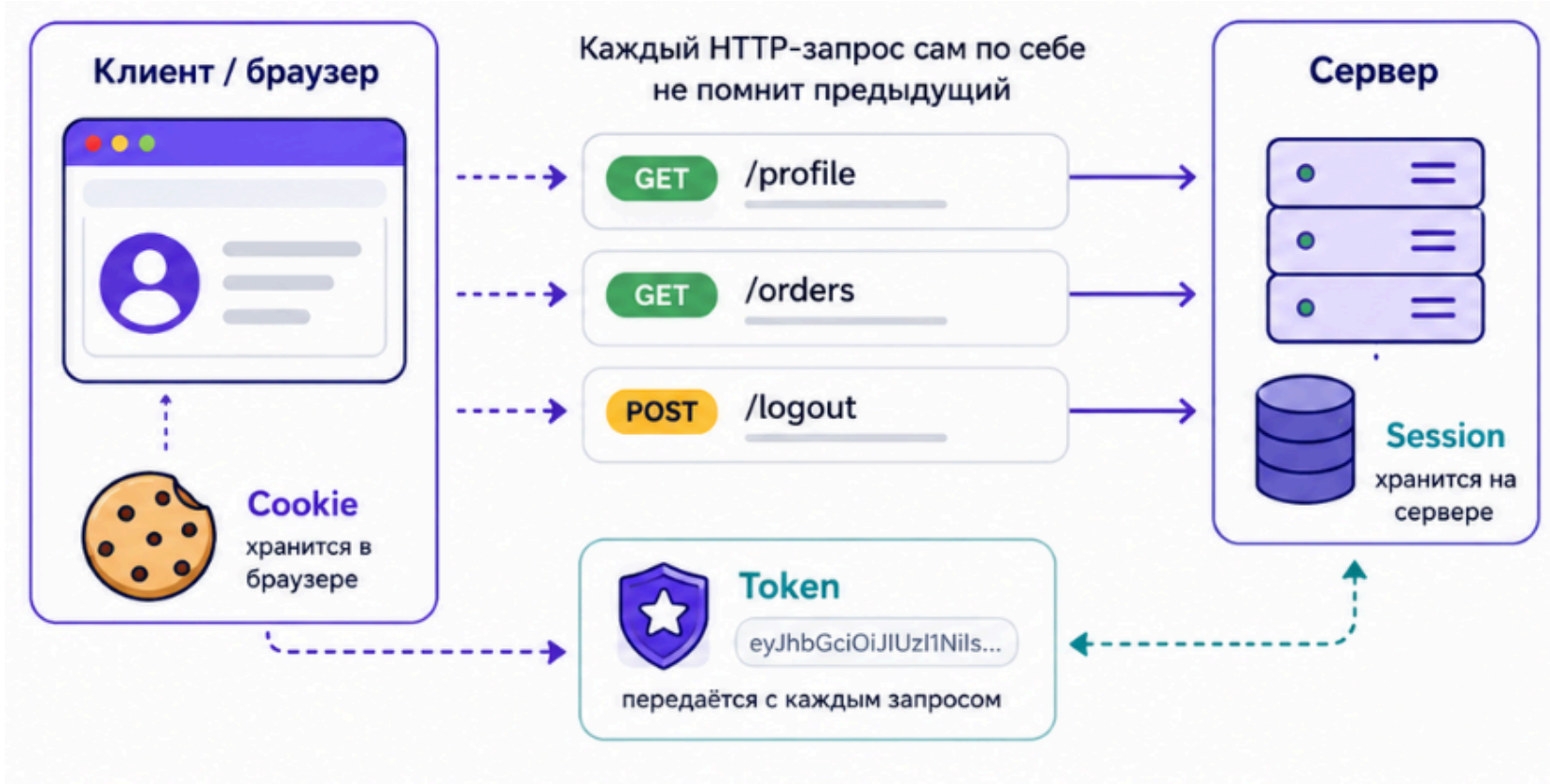
5xx — Ошибки сервера
Сервер не смог выполнить запрос корректно.
Примеры: 500, 502, 503, 504

Забавные статусы-кодов

В чем разница между 401 и 403?



Как веб всё-таки «помнит» пользователя



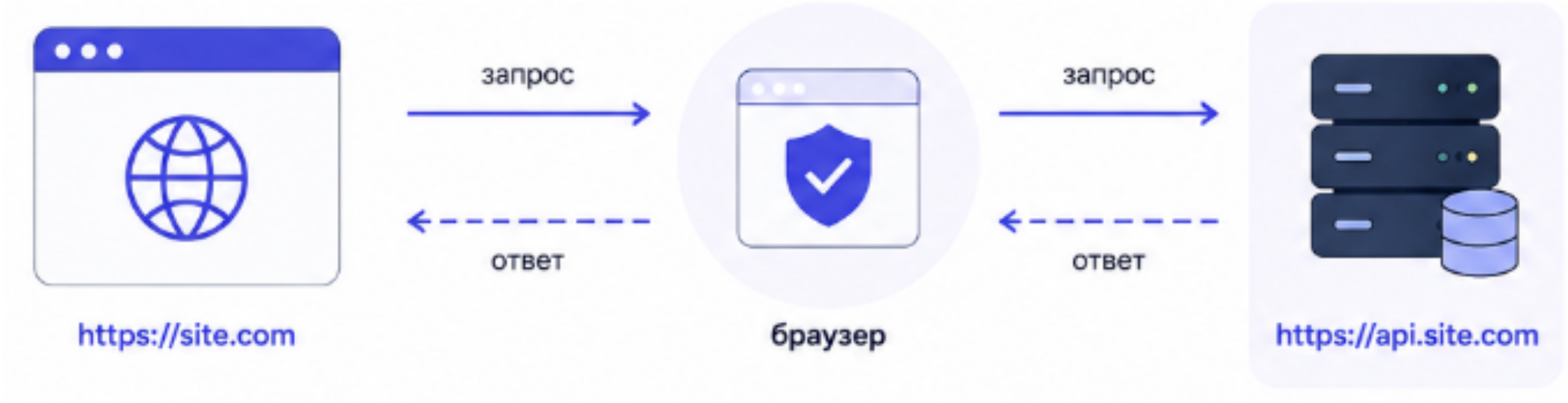
Cookie
браузер хранит маленькие данные и отправляет их с запросами.

Session
сервер хранит состояние и связывает его с session_id.

Token
клиент передаёт токен, сервер проверяет подпись или claims.

CORS: что это и зачем он нужен?

CORS (Cross-Origin Resource Sharing) - это механизм браузера, который разрешает или запрещает странице обращаться к ресурсам другого origin.



Same-origin — всё ок

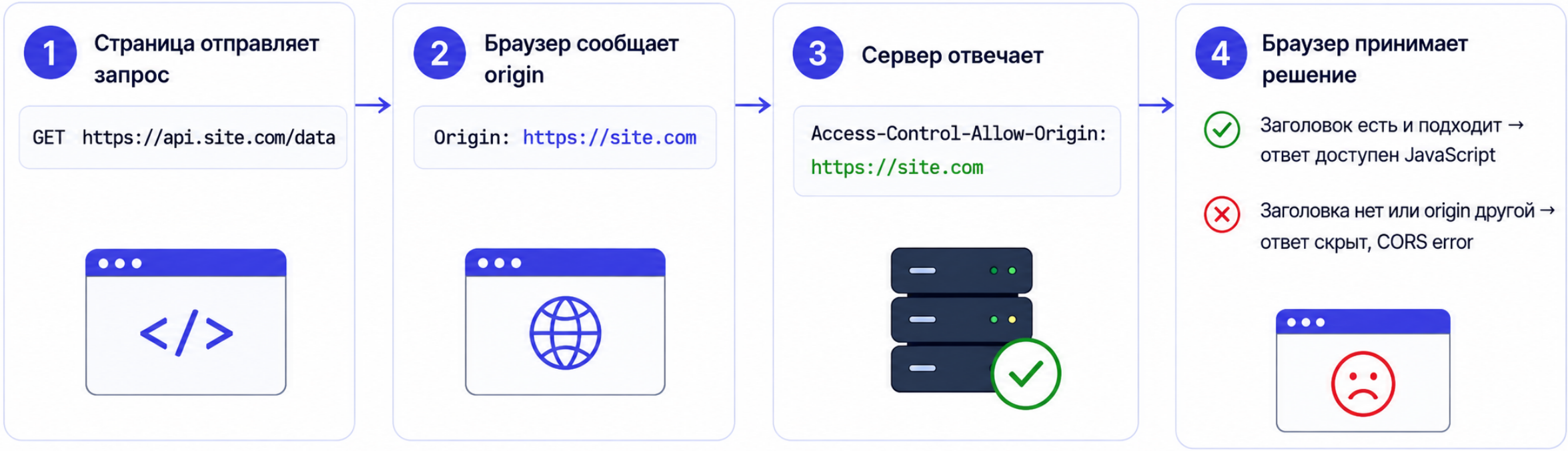
Страница и API имеют один origin — браузер отдаёт ответ.

Cross-origin — нужно разрешение

Если origin другой — сервер должен явно разрешить доступ через CORS-заголовки.

Почему возникает CORS error?

CORS error возникает не потому, что сервер обязательно сломан.
При запросе к другому origin браузер проверяет, разрешил ли сервер странице читать ответ.
Если нужного разрешения нет, сервер может даже вернуть 200 OK, но браузер не передаст данные JavaScript.



Что видно в DevTools Network

Жизненный цикл запроса

Elements | Console | Sources | **Network** | Performance | Memory | Application | Security | Lighthouse

Preserve log | Disable cache | No throttling

Filter | Invert | Hide data URLs | All | Fetch/XHR | Doc | CSS | JS | Img | Media | Font | WS | Wasm | Other

Name	URL	Method	Status	Type	Initiator	Size	Time	Waterfall
api/user	https://api.example.com/api/user	GET	200	xhr	app.js:120	1.2 kB	87 ms	
profile	https://example.com/profile	GET	200	document	Other	24.6 kB	210 ms	
styles.css	https://example.com/assets/styles.css	GET	200	stylesheet	(index):5	12.1 kB	45 ms	
app.js	https://example.com/assets/app.js	GET	200	script	(index):6	45.3 kB	62 ms	
avatar.png	https://example.com/images/avatar.png	GET	200	png	(index):7	8.4 kB	38 ms	

URL — Полный адрес запрашиваемого ресурса

Method — Метод запроса (GET, POST, PUT и др.)

Status — HTTP-статус ответа сервера (200, 404, 500...)

Headers — Заголовки запроса и ответа (cookies, content-type и др.)

Payload — Тело запроса (данные, которые вы отправляете)

Response — Тело ответа (данные от сервера)

Timing — Расшифровка времени каждого этапа запроса

Size — Размер ответа (сжатый / несжатый)

Детали запроса (пример)

Headers | Payload | Response | Timing

General

Request URL:	https://api.example.com/api/user
Request Method:	GET
Status Code:	200 OK
Remote Address:	93.184.216.34:443
Referrer Policy:	strict-origin-when-cross-origin

Response Headers

content-type:	application/json; charset=utf-8
cache-control:	no-cache
date:	Mon, 06 May 2024 10:15:30 GMT
content-length:	1256

Что видно в DevTools Network